📚 Python Dictionary-All Topics(At a Glance)

1. Basics

- **Definition**: A collection of key-value pairs, unordered, mutable.
- **Syntax**:

```
my_dict = {"name": "Alice", "age": 25, "city": "Dhaka"}
```

Empty Dictionary:

```
Empty1 = {}
print(type(empty1)) #<class 'dict'>
empty2 = dict()
print(type(empty2)) #<class 'dict'>
```

2. Dictionary Keys and Values

- **Keys must be unique & immutable** (strings, numbers, tuples).
- **Values can be any type** (mutable or immutable).
- Examples:

```
{"a": 1, "b": [1,2,3], "c": {"nested": True}}
```

✅ Dictionary Key কী কী ধরনের হতে পারে?

Dictionary key অবশ্যই **immutable (পরিবর্তনযোগ্য নয়)** ডাটা টাইপ হতে হবে।
যেমন:

- **String**

```
{"name": "Nusrat"}
```

- **Number (int, float)**

```
{1: "One", 3.14: "pi"}
```

- **Tuple** (যদি tuple-এর ভেতরের সব উপাদান immutable হয়। Tuple immutable data type)

```
{(1, 2): "Point A", (3, 4): "Point B"}
```

- **Boolean**

```
{True: "Yes", False: "No"}
```

✕ Dictionary Key কী কী ধরনের হতে পারে না?

Key যদি **mutable (পরিবর্তনযোগ্য)** হয়, তবে dictionary-তে ব্যবহার করা যাবে না।
কারণ mutable data পরিবর্তন হলে hash value বদলে যায়, এতে dictionary ঠিকমতো কাজ করবে না।

যা key হতে পারবে না:

- **List**

```
{[1, 2, 3]: "Invalid"}   # ✕ Error
```

- **Set**

```
{{1, 2, 3}: "Invalid"}   # ✕ Error
```

- **Dictionary** (nested dictionary key হিসেবে ব্যবহার করা যাবে না)

```
{{"a": 1}: "Invalid"}    # ✕ Error
```

📝 মনে রাখবেন:

- **Key immutable হতে হবে।**
- **Value যেকোনো কিছু হতে পারে (mutable বা immutable উভয়ই)।**

☞ উদাহরণ:

```
valid_dict = {
    "name": "Nusrat",
    10: [1, 2, 3],         # value list হতে পারে
    (1, 2): {"x": 5},      # value dictionary হতে পারে
}
```

⬘ হ্যাশ টেবিল কী?

হ্যাশ টেবিল হলো একটা **ডেটা স্ট্রাকচার**, যেখানে তথ্য (data) **কী–ভ্যালু জোড়া** আকারে রাখা হয়।

☞ Python-এর **dictionary (dict)** আসলে ভেতরে ভেতরে **hash table** দিয়েই বানানো।

---

১. মূল ধারণা

- তুমি যদি একটা **key** দাও (যেমন `"name"`),
- Python সেই key-এর উপর **hash function** চালায় → একটা সংখ্যা তৈরি হয় (hash value)।
- সেই সংখ্যা একটা **index (ঘর নম্বর)** ঠিক করে দেয় একটা বড় array (ঘরগুলির) ভেতরে।
- তারপর value (যেমন `"Alice"`) সেই index-এ রাখা হয়।

☞ তাই পরে তুমি যখন `d["name"]` খুঁজবে, Python আবার hash করবে এবং সরাসরি সেই ঘরে চলে যাবে।
⚡ এজন্যই dictionary তে খোঁজাখুঁজি খুব দ্রুত (O(1) টাইমে)।

---

২. সহজ উদাহরণ (চিঠির বাক্স)

ভাবো তোমার কাছে ১০টা চিঠির বাক্স আছে, ০–৯ পর্যন্ত নাম্বার লেখা।

এখন তুমি রাখতে চাইছো:

```
{"name": "Alice"}
```

১. `"name"` এর hash ধরো হলো `12345`।
২. এবার array সাইজ ১০ দিয়ে ভাগ করলে → `12345 % 10 = 5`।
৩. `"Alice"` রাখা হলো **বাক্স ৫ নম্বরে**।

যখন `"name"` খুঁজবে → আবার hash হবে → আবার ৫ নম্বর বাক্সে গিয়ে `"Alice"` পাওয়া যাবে। ✅

---

৩. হ্যাশ কোলিশন (Collision)

যদি দুইটা key একই index-এ পড়ে যায়, তখন একে বলে **collision**।

যেমন:

```
{"name": "Alice", "course": "CS"}
```

ধরা যাক `"name"` আর `"course"` দুটোই ৫ নম্বর বাক্সে জমা পড়ল।

⚡ তখন Python বিশেষ কৌশল (open addressing + probing) ব্যবহার করে দুটোই নিরাপদে রেখে দেয়।

---

৪. হ্যাশ টেবিলের কাজ

- **Insert (যোগ করা)** → hash বের করে index-এ value রাখা।
- **Search (খোঁজা)** → hash বের করে index-এ গিয়ে value নেওয়া।
- **Delete (মুছা)** → hash বের করে index-এ গিয়ে value মুছে ফেলা।

☞ সব কাজ গড়ে **খুব দ্রুত (O(1))** হয়।

---

৫. Python Dictionary-তে ব্যবহার

```
d = {}
d["name"] = "Alice"    # সংরক্ষণ করা
print(d["name"])       # খুঁজে বের করা
```

- `"name"` hash হয়ে index পায় → সেখানে `"Alice"` রাখা হয়।
- পরে খুঁজতে গেলে সরাসরি index-এ গিয়ে পেয়ে যায়।

---

৬. ছোট্ট টেবিল আকারে

Key	Hash(Key)	Index (mod 10)	Stored Value
"name"	12345	5	"Alice"
"course"	54325	5 (collision)	"CS"

☞ Python collision হলে ভেতরে ভেতরে সামলে নেয়।

ধরো, একটা **হ্যাশ টেবিল** অনেকটা বড়ো ড্রয়ারওয়ালা আলমারির মতো।
প্রতিটি key (চাবি) কে hash function একটা নির্দিষ্ট ড্রয়ারে রাখার ঠিকানা দেয়।

সমস্যা:

ধরা যাক, দুটি key-র hash value একই হলো।
মানে — দুটো জিনিসই একই ড্রয়ারে রাখতে হবে। এখন কী করা হবে?

সমাধান – Open Addressing + Probing

Python (এবং অনেক ভাষা) তখন একটা কৌশল নেয়:

1. প্রথম ড্রয়ারটা যদি ব্যস্ত থাকে (আগেই কারো জায়গা হয়ে গেছে),
2. তাহলে সে পাশের খালি ড্রয়ার চেক করে।
3. আবার সেটা ভরা থাকলে, পরেরটা দেখে… এভাবে চলতে থাকে।

এই ধারাবাহিক খোঁজার প্রক্রিয়াকেই বলে **Probing**।
আর যখন একই টেবিলের ভেতরে অন্য কোনো খালি জায়গায় (ড্রয়ারে) রাখে, সেটাকে বলে **Open Addressing**।

উদাহরণ:

- Key `A` আর Key `B` — দুজনের hash মান হলো 7।
- টেবিলের 7 নাম্বার ড্রয়ারে `A` আগে বসে গেছে।
- এখন `B` এলো। 7 নাম্বার ড্রয়ার ব্যস্ত।
- Python তখন **পরের ফাঁকা ড্রয়ার** খুঁজে নেয় (যেমন 8 বা 9…) এবং `B` কে সেখানে রাখে।

☞ এভাবে Python dictionary-তে একই hash হলে (collision) ও ডাটা হারায় না।

dictionary faster নাকি list/array faster?

১)Dictionary (Hash Table ভিত্তিক)

- Dictionary-তে key দিয়ে খোঁজা হয়।
- ভেতরে hash table থাকে → মানে key এর উপর hash function চালিয়ে সরাসরি slot (address) পাওয়া যায়।
- তাই, **lookup, insert, delete** → গড়ে সময় লাগে **O(1)** (constant time)।

☞ উদাহরণ:

```
d = {"apple": 10, "orange": 20}
print(d["orange"])   # একেবারে সাথে সাথে value পাওয়া যাবে
```

---

২) List / Array

- List/array-তে index দিয়ে খোঁজা হলে দ্রুত (O(1)) পাওয়া যায়।
- `a = [5, 10, 20]`
- `print(a[2])   # খুব দ্রুত, কারণ index দিয়ে সরাসরি খুঁজে নেয়`
- কিন্তু **value দিয়ে খোঁজা** (যেমন 20 আছে কিনা) করতে হলে পুরো লিস্ট ঘেঁটে দেখতে হয় → সময় লাগে **O(n)**।
- `20 in a   # সব element একে একে চেক করতে হবে`

⚡ তুলনা (Comparison)

- **Index দিয়ে খোঁজা** → List/Array দ্রুত।
- **Key দিয়ে খোঁজা** → Dictionary অনেক দ্রুত (list থেকে অনেক এগিয়ে)।
- **Value দিয়ে খোঁজা** → Dictionary-তেও O(n), list-এর মতোই।

📌 সহজভাবে মনে রাখো

- যদি **position (index)** আগে থেকে জানা থাকে → **List/Array** সেরা।
- যদি **কোনো নাম/চাবি (key)** দিয়ে ডাটা খুঁজতে হয় → **Dictionary** অনেক দ্রুত।

🔑 Hash Function কী?

☞ **Hash Function** হলো এমন একটা ফাংশন,
যা **যেকোনো data (string, number, file, ইত্যাদি)** কে ইনপুট হিসেবে নিয়ে
একটা **fixed size সংখ্যা (hash value)** আউটপুট দেয়।

এটা অনেকটা ডেটার **ডিজিটাল ফিঙ্গারপ্রিন্ট** এর মতো।

□ বৈশিষ্ট্য

1. **Deterministic** → একই ইনপুট দিলে সবসময় একই আউটপুট দেবে।
   যেমন: `"apple"` দিলে সবসময় একই hash মান আসবে।
2. **Fast** → hash মান বের করা খুব দ্রুত হয়।
3. **Fixed size** → ইনপুট যত বড়ই হোক (১ KB বা ১ GB), আউটপুট সবসময় নির্দিষ্ট আকারের হবে।
4. **Uniform distribution** → চেষ্টা করে মানগুলোকে সমানভাবে ছড়িয়ে দিতে (collisions কমানোর জন্য)।

□ Hash Function এর ব্যবহার

- **Python Dictionary** → key কে hash করে টেবিলে slot বের করে।
- **Databases** → দ্রুত খোঁজার জন্য।
- **Cryptography** → ডেটা সুরক্ষার জন্য (যেমন SHA256)।
- **File systems** → ডুপ্লিকেট ফাইল খুঁজে বের করার জন্য।

□ Python এ Hash Function উদাহরণ

Python এ built-in `hash()` আছে।

```
print(hash("apple"))
print(hash("orange"))
print(hash(12345))
```

☞ প্রতিবার রান করলে মান আলাদা হতে পারে (Python নিরাপত্তার কারণে random seed ব্যবহার করে)।
কিন্তু একই রান এর ভেতরে একই key এর hash মান সবসময় একই থাকে।

একই রানের ভেতরে মানে কী?

"**একই রান**" বলতে বোঝায় — যখন তুমি Python প্রোগ্রাম একবার চালাও, তখন যে execution চলছে সেটা হলো *একটা run* বা *একটা session*।

উদাহরণ:

ধরো তুমি Python shell খুললে বা একটা প্রোগ্রাম চালালে:

```
print(hash("apple"))
print(hash("apple"))
```

☞ এই দুইবার `hash("apple")` একই **run/session**–এর ভেতরে হচ্ছে।
তাই দুইবারেই একই মান আসবে।

কিন্তু যখন নতুনভাবে Python চালাও:

- তুমি Python বন্ধ করে আবার চালালে
- অথবা নতুন script run করলে

তাহলে Python শুরুতে **random seed** সেট করে, যার কারণে `hash("apple")` এবার অন্য সংখ্যা দিতে পারে।

📌 সারকথা:

- **একই run/session** → hash("apple") সবসময় একই মান দেবে।
- **ভিন্ন run/session** → hash("apple") এর মান বদলাতে পারে (security কারণে)।

hash("apple") % 10 → 7

hash("orange") % 10 → 7 (collision!)

hash("banana") % 10 → 3

☞ "apple" আর "orange" দুটোই 7 এ পড়ছে → collision।

3. Accessing Elements
student={

"name": “Alice”,

"age": 20,

"department": “CSE”,

"semester": “4th”

}

- By key:

```
my_dict["name"]   # Alice
```

- Using `.get()` (avoids error if key missing):

```
student.get("age")    # 20
student.get("salary")  # None
print(student.get("age", "Not Found”)#20
print(student.get("salary", "Not Found"))#Not Found
```

3. Accessing Elements

- By key:
- `my_dict["name"]   # Alice`
- Using `.get()` (avoids error if key missing):
- `my_dict.get("age")    # 25`
  `my_dict.get("salary") # None`
  `my_dict.get("salary", “Not Found”) # Not Found`

4. Adding & Updating

- Add new key-value:
- `student["country"] = "Bangladesh"`
- Update existing:

  ```
  student["age"] = 26
  ```

5. Removing Elements

- `pop(key)` → removes key and returns value.
- `popitem()` → removes last inserted pair.
- `del` keyword → removes by key.
- `clear()` → empties dictionary.
- `my_dict.pop("city")`
- `del my_dict["age"]`
  `my_dict.clear()`

6. Dictionary Methods

- `keys()` → all keys
- `values()` → all values
- `items()` → key-value pairs

print(student.keys()) # dict_keys(['name', 'age', 'department', 'semester'])
print(student.values()) # dict_values(['Alice', 20, 'CSE', '4th'])

print(student.items())#dict_items([('name', 'Alice'), ('age', 20), ('department', 'CSE'), ('semester', '4th')])

- `update()` → merge with another dict
- Example:
- `d1 = {"a":1, "b":2}`
- `d2 = {"c":3}`
- `d1.update(d2)  # {"a":1,"b":2,"c":3}`

কিন্তু যদি সেইম কী হয়, তাহলে ডাটা রিপ্লেইস হয়ে যাবে। উদাহরণঃ

```
student={

    "name": "Alice",

    "age": 20,

    "department": "CSE",

    "semester": "4th"

}

student2={

    "data":"Another",
```

```
    "profession": "Student"
}
student.update(student2)
print(student)# {'name': 'Alice', 'age': 20, 'department': 'CSE', 'semester': '4th', 'data': 'Another', 'profession': 'Student'}
```

7. Looping

```
my_dict={
    "apple":10,
    "banana":20,
    "grape":30
}
```

- **Keys**:

```
for k in my_dict:
    print(k)
for k in my_dict.keys():
    print(k)
```

Values:

```
for v in my_dict.values():
    print(v)
```

Items:

```
for k, v in my_dict.items():
    print(k, v)
```

8. Dictionary Comprehension

- Syntax:
- `{key: value for key, value in iterable}`

- Example:

```
squares = {x: x*x for x in range(5)}
```

🔑 Dictionary Comprehension (ডিকশনারি কমপ্রিহেনশন)

1. Dictionary Comprehension কী?

এটা হলো এক লাইনে dictionary তৈরি করার শর্টকাট সিনট্যাক্স।
☞ যেমন আমরা list comprehension ব্যবহার করি, তেমনভাবেই dictionary comprehension ব্যবহার করা যায়।

2. সিনট্যাক্স

```
{key_expression : value_expression for item in iterable if condition}
```

- **key_expression** → key কেমন হবে
- **value_expression** → value কেমন হবে
- **iterable** → কোন sequence থেকে আসবে (যেমন list, range ইত্যাদি)
- **condition** (optional) → শর্ত দিয়ে filter করা যায়

3. উদাহরণ

(ক) সাধারণ dictionary comprehension

```
squares = {x: x**2 for x in range(5)}
print(squares)
```

☞ Output:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

(খ) শর্ত (condition) সহ

```
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares)
```

☞ Output:

```
{0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

(গ) list থেকে dictionary বানানো

```
names = ["Nusrat", "Korayil", "AI"]
lengths = {name: len(name) for name in names}
print(lengths)
```

☞ Output:

```
{'Nusrat': 6, 'Korayil': 7, 'AI': 2}
```

(ঘ) Nested dictionary comprehension

```
table = {x: {y: x*y for y in range(1, 6)} for x in range(1, 6)}
print(table[3])
```

☞ Output:

```
{1: 3, 2: 6, 3: 9, 4: 12, 5: 15}
```

4. কেন ব্যবহার করবেন?

- কোড **সংক্ষিপ্ত** হয়
- **পড়তে সহজ** হয়
- For loop লিখে dictionary বানানোর চেয়ে দ্রুত

✍ সারসংক্ষেপ:
Dictionary comprehension = এক লাইনে dictionary তৈরির কৌশল।
☞ `{key: value for item in iterable if condition}`

3)Problem 3: String Character Frequency

Task:
`text = "hello world"` এর জন্য dictionary বানান যেখানে

- **key = character**, **value = character count**
- spaces বাদ দিন

text="hello world"

freq={char:text.count(char) for char in text if char!=" "}

print(freq)# {'h': 1, 'e': 1, 'l': 3, 'o': 2, 'w': 1, 'r': 1, 'd': 1}

9. Nested Dictionaries

- Example:
- `students = {`
- `    "s1": {"name": "Nusrat", "age": 28},`
- `    "s2": {"name": "Korayil", "age": 30}`
- `}`
- Access:

```
students["s1"]["name"]  # Nusrat
```

10. Dictionary vs Other Data Types

- **Dictionary vs List**
  - Dict uses keys, list uses index.
- **Dictionary vs Set**
  - Dict stores key-value, set stores only keys (unique).

11. Advanced Features

- **Default dictionary (`collections.defaultdict`)**

```
from collections import defaultdict
d = defaultdict(int)
d["x"] += 1
```

- **Ordered dictionary (`collections.OrderedDict`)**
  Keeps insertion order (from Python 3.7+, normal dict also keeps order).
- **Counter (as dict subclass)**
- `from collections import Counter`
- `Counter("banana")  # {'b':1,'a':3,'n':2}`
- **dict.fromkeys()**
- `keys = ["a", "b", "c"]`
- `dict.fromkeys(keys, 0)   # {'a':0,'b':0,'c':0}`

---

12. Dictionary Packing & Unpacking

- Packing:
- `dict1 = dict(name="Alice", age=25)`
- Unpacking:
- `d = {"a":1, "b":2}`
-
```
print(*d)       # a b
print(**d)      # keyword unpacking
```

print(**d) সরাসরি চালালে `error` দেবে।
⍰ `**` কী করে?

- Python–এ `**` ব্যবহার হয় **dictionary unpacking**–এর জন্য।
- মানে, dictionary–র key–value–গুলোকে function–এর **keyword arguments** হিসেবে পাঠায়।

উদাহরণ:

```
def greet(name, age):
    print(f"My name is {name}, age {age}")

info = {"name": "Nusrat", "age": 28}

greet(**info)
```

☞ এখানে `**info` মানে:

```
greet(name="Nusrat", age=28)
```

● কেন `print(**d)` error দেয়?

কারণ `print()` function–এর মধ্যে এমন কোনো parameter নেই যেটা `"apple"`, `"banana"` ইত্যাদি key হিসেবে নিতে পারে।
তুমি dictionary–র key–গুলো function argument হিসেবে পাঠাতে চাইছো, কিন্তু `print()` ওগুলো চিনতে পারে না। তাই error আসে।

□ তাহলে dictionary unpacking ঠিকভাবে দেখতে চাইলে?

```
d = {"a": 1, "b": 2}

print(*d)       # শুধু key গুলো print করবে → a b
print(d)        # পুরো dict print করবে → {'a': 1, 'b': 2}
print(*d.items())  # tuple আকারে print করবে → ('a', 1) ('b', 2)
```

✅ সারকথা:

- `*d` → keys বের করে
- `**d` → keys–কে keyword argument বানায় (শুধু function call–এ কাজ করে)
- `print(**d)` কাজ করবে না, কারণ `print()` function ওই keys–কে argument হিসেবে নিতে জানে না।

13. Dictionary in Functions

- Passing as argument:
- `def show_info(**kwargs):`
- `    for k,v in kwargs.items():`
  `    print(k, v)`

```
show_info(name="Nusrat", age=28)
```

Ex-01: student={

"name":"Nusrat",

"department":"CSE"

}

def greet(name, department):

print("Hello dear",name,"from ",department, "department")

greet(**student)

Ex-02: student={

"name":"Nusrat",

"department":"CSE"

}

def greet2(name, department):

for k,v in student.items():

print(k,":",v)

greet2(**student)

14. Dictionary Use Cases

- Counting frequencies
- Fast lookups (O(1) average time)
- Representing JSON-like structures
- Mapping & configuration data